

EXPERIMENT 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
 - o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
 - o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
 - o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
 - o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output:

Program #1: Python program that uses the hashlib library to create the hash of a given string:

Code:

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()

    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))

    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()

    # Return the hash string
    return hash_string

input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

Output:

```
Enter a string: janhavi
Hash: 64f21bd3afee1e7210bbf9465d63af514de1b3be23ab751686017db7652d9520
```

Program #2: Program to generate required target hash with input string and nonce

Code:

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")
```

```

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)

```

Output:

```

Enter a string: janhavi
Enter the nonce: 78
Hash Code:
02e2408403b919a84133e691d90d869161844a3da74357dda57bb3421773f608

```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string**Code:**

```

import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)

        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

```

```

# Get user input
input_string = "Janhavi"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)
# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)

```

Output:

```

Hash: 041118d9780230a6ac9a168d2b3b2295c8a1e09c1252530a32e9a20ff572acea
Input String: Janhavi
Leading Zeros: 1
Expected Nonce: 43

```

Program 4: Generating Merkle Tree for given set of Transactions**Code:**

```

import hashlib

def sha256(data):
    return hashlib.sha256(data.encode("utf-8")).hexdigest()

def build_merkle_tree(transactions):

    if len(transactions) == 0:
        print("No transactions!")
        return None

    print("===== ORIGINAL TRANSACTIONS =====")

    for i, transaction in enumerate(transactions, 1):
        print(f"Transaction {i}: {transaction}")

    print("\n===== HASHING TRANSACTIONS =====")

    current_level = []

```

```

for i, transaction in enumerate(transactions, 1):
    hashed = sha256(transaction)
    current_level.append(hashed)

    print(f"\nTransaction {i}: {transaction}")
    print(f"Hash {i}: {hashed}")
step = 3

while len(current_level) > 1:

    print(f"\n===== STEP {step} =====")

    for i, item in enumerate(current_level, 1):
        print(f"{i}. {item}")

    if len(current_level) % 2 != 0:
        print("\nNumber of items is ODD.")
        print("Duplicating the last hash...")
        current_level.append(current_level[-1])
        print(f"Duplicated: {current_level[-1]}")

    print("\n===== HASHING PAIRS =====")

    new_level = []

    for i in range(0, len(current_level), 2):

        left = current_level[i]
        right = current_level[i + 1]

        print("\nPair:")
        print("Left :", left)
        print("Right:", right)

        combined = left + right

        print("Combined:")
        print(combined)

        hashed = sha256(combined)

```

```

        print("SHA-256:")
        print(hashed)

        new_level.append(hashed)

    current_level = new_level

    print("\n===== NEXT LEVEL =====")

    for i, item in enumerate(current_level, 1):
        print(f"Hash {i}: {item}")

    step += 1

merkle_root = current_level[0]

print("\n===== FINAL STEP =====")
print("Merkle Root:")
print(merkle_root)

return merkle_root

transactions = [
    "Alice -> Bob : $200",
    "Bob -> Dave : $500",
    "Dave -> Eve : $100",
    "Eve -> Alice : $300",
    "Roo -> Bob : $50"
]
merkle_root = build_merkle_tree(transactions)

print("\n=====")
print("FINAL MERKLE ROOT:")
print(merkle_root)
print("=====")

```

Output:

```

===== ORIGINAL TRANSACTIONS =====
Transaction 1: Alice -> Bob : $200

```

Transaction 2: Bob -> Dave : \$500
Transaction 3: Dave -> Eve : \$100
Transaction 4: Eve -> Alice : \$300
Transaction 5: Roo -> Bob : \$50

===== HASHING TRANSACTIONS =====

Transaction 1: Alice -> Bob : \$200
Hash 1: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760

Transaction 2: Bob -> Dave : \$500
Hash 2: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46

Transaction 3: Dave -> Eve : \$100
Hash 3: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da

Transaction 4: Eve -> Alice : \$300
Hash 4: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a

Transaction 5: Roo -> Bob : \$50
Hash 5: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

===== STEP 3 =====

1. f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
2. 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
3. 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
4. cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
5. cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Number of items is ODD.

Duplicating the last hash...

Duplicated:

cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

===== HASHING PAIRS =====

Pair:

Left : f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760

Right: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46

Combined:

f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760768890d3b5
8d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46

SHA-256:

471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9

Pair:

Left : 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da

Right: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a

Combined:

43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890dacba341d796

5fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a

SHA-256:

a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

Pair:

Left : cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Right: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Combined:

cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58accfe1dc26c7

c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

SHA-256:

314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

===== NEXT LEVEL =====

Hash 1: 471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9

Hash 2: a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

Hash 3: 314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

===== STEP 4 =====

1. 471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9

2. a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

3. 314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Number of items is ODD.

Duplicating the last hash...

Duplicated:

314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

===== HASHING PAIRS =====

Pair:

Left : 471608d3d6f2a642ff3a84dfaeeaebbb1c271a9aed5b37a82d61c4784558e80e9

Right: a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

Combined:

471608d3d6f2a642ff3a84dfaeeaebbb1c271a9aed5b37a82d61c4784558e80e9a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

SHA-256:

Ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee

Pair:

Left : 314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Right: 314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Combined:

314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

SHA-256:

6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

===== NEXT LEVEL =====

Hash 1: ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee

Hash 2: 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

===== STEP 5 =====

1. ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee

2. 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

===== HASHING PAIRS =====

Pair:

Left : ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee

Right: 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

Combined:

ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

SHA-256:

96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

===== NEXT LEVEL =====

Hash 1: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

===== FINAL STEP =====

Merkle Root:

96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

=====

FINAL MERKLE ROOT:

96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

=====

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management